

KUBERNETES

Finalidad de Kubernetes subir nuestros proyectos web y tareas de administración, etc. Hacer que los dominios (Distintas páginas) apunten a la VPS y con el proxy manager redireccionarlos a puertos distintos y que tengan alta disponibilidad.

La razón por la que se usará MicroK8s es porque si usamos la distribución k3d (versión en desarrollo) No es un kubernetes real ya que si se cae la VPS perdemos directamente el proyecto y los pods. El MicroK8s nos permite gastar menos recursos para poder hacer el proyecto en nuestra VPS.

1 pod → equivale a un Docker compose que realizamos.

En el control plane contiene una etcd (base de datos), una API (basada en NEST, etc), lo que no debe contener bajo ningún concepto en el control plane son Pods ya que eso lo llevará los workers (Nodes).

INSTALACIÓN

Para instalar kubernetes necesitamos 4 máquinas. 1 control plane y 3 nodos, en el que 2 nodos trabajan y la tercera sirve de reserva para que cuando se caiga. (LA PRACTICA GUAPA SERÍA TENER 4 VPS DE CONTABO).

Para la instalación deberemos instalar los paquetes que deberán contener un control plane y los workers unirlos a un cluster.

Con el MicroK8s permite que el control plane sirva también con un worker ahorrando así una máquina (No recomendado, pero es lo que haremos en clase).

PASOS DE LA INSTALACIÓN (MICROK8S)

Actualizamos el sistema

```
jiahaolin@vmi2812274:~$ sudo apt update && sudo apt upgrade -y
```

Desactivamos la swap de linux

```
jiahaolin@vmi2812274:~$ sudo swapoff -a
```

Modificamos una cosa en el fstab, básicamente comentamos la línea para que no se active de nuevo automáticamente

```
sudo sed -i ' / swap / s/^/#/' /etc/fstab
```

```
jiahaolin@vmi2812274:~$ sudo sed -i ' / swap / s/^/#/' /etc/fstab
```

INSTALAMOS MicroK8s

```
Sudo snap install microk8s --classic
```

```
jiahaolin@vmi2812274:~$ sudo snap install microk8s --classic
2026-01-14T11:00:35+01:00 INFO Waiting for automatic snapd restart...
microk8s (1.32/stable) v1.32.9 from Canonical✓ installed
```

Comprobamos si con la instalación se crea el grupo microk8s

```
Cat /etc/group
```

```
docker:x:988:jiahaolin
jiahaolin:x:1000:
jiahaolin:x:1001:
microk8s:x:1002:
```

Añadimos nuestro usuario al grupo microk8s

```
Sudo usermod -aG microk8s $USER
```

```
jiahaolin@vmi2812274:~$ sudo usermod -aG microk8s $USER
```

Comprobamos que se haya añadido

```
microk8s:x:1002:jiahaolin
```

Luego ejecutamos

Newgrp microk8s para poder ejecutar el siguiente comando

```
jiahaolin@vmi2812274:~$ newgrp microk8s
```

Microk8s status --wait-ready

```
jiahaolin@vmi2812274:~$ microk8s status --wait-ready
microk8s is running
high-availability: no
datastore master nodes: 127.0.0.1:19001
datastore standby nodes: none
addons:
  enabled:
    dns                # (core) CoreDNS
    ha-cluster         # (core) Configure high availability on the current node
    helm               # (core) Helm - the package manager for Kubernetes
    helm3              # (core) Helm 3 - the package manager for Kubernetes
```

Habilitamos estos servicios

microk8s enable dns ingress storage metrics-server

```
jiahaolin@vmi2812274:~$ microk8s enable dns ingress storage metrics-server

Enabling metrics-server
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-reader created
clusterrole.rbac.authorization.k8s.io/system:metrics-server created
rolebinding.rbac.authorization.k8s.io/metrics-server-auth-reader created
clusterrolebinding.rbac.authorization.k8s.io/metrics-server:system:auth-delegator created
clusterrolebinding.rbac.authorization.k8s.io/system:metrics-server created
service/metrics-server created
deployment.apps/metrics-server created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
clusterrolebinding.rbac.authorization.k8s.io/microk8s-admin created
Metrics-Server is enabled
```

Y vemos que se nos haya generado unos pods con el siguiente comando.

microk8s kubectl get pods -A

```
Metrics-Server is enabled
jiahaolin@vmi2812274:~$ microk8s kubectl get pods -A
NAMESPACE   NAME                                                                 READY   STATUS    RESTARTS   AGE
ingress      nginx-ingress-microk8s-controller-5zcd9                          1/1     Running   0          63s
kube-system  calico-kube-controllers-5947598c79-mtcrq                         1/1     Running   0          47m
kube-system  calico-node-szsb2                                                1/1     Running   0          47m
kube-system  coredns-79b94494c7-wp62p                                         1/1     Running   0          47m
kube-system  hostpath-provisioner-c778b7559-dh54s                             1/1     Running   0          60s
kube-system  metrics-server-7dbd8b5cc9-k28v1                                  1/1     Running   0          57s
```

Ahora instalamos el kubectl en nuestra VPS, no queremos usar el de microk8s, ya que eso nos limitaría solamente a poder conectarnos a esa kubernetes. Al instalarlo de manera global podemos conectarnos a otras kubernetes.

Sudo snap install kubectl --classic

```
jiahaolin@vmi2812274:~$ sudo snap install kubectl --classic
kubectl 1.34.3 from Canonical✓ installed
```

```
jiahaolin@vmi2812274:~$ ls -a
. .bash_history .bashrc .docker .gitconfig .lessht .profile .sudo_as_admin_successful .wget-hsts examen jiahaohlc megacrossover
.. .bash_logout .cache .dotnet .kube .local .ssh .vscode-server devops gitremote kubernetes snap
jiahaolin@vmi2812274:~$ cd .kube
jiahaolin@vmi2812274:~/.kube$ ls
cache
jiahaolin@vmi2812274:~/.kube$
```

Ahora creamos un fichero config para cada kubernetes que tengamos que en este caso es solamente 1.

Microk8s config > ~/.kube/config

```
jiahaolin@vmi2812274:~/.kube$ microk8s config > ~/.kube/config
```

```
jiahaolin@vmi2812274:~/.kube$ ls
cache config
```

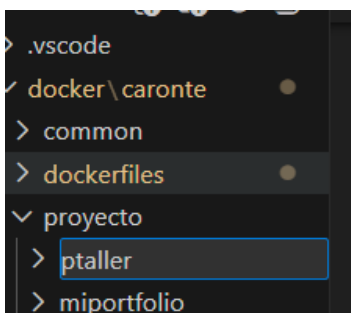
Este archivo contiene el certificado, nombre del servidor, el servidor, etc. básicamente toda la configuración de un kubernetes que en este caso nos pasa el clúster del microk8s.

Podemos observar un nodo que se ha creado.

kubectl get nodes -o wide

```
jiahaolin@vmi2812274:~/.kube$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
vmi2812274    Ready     <none>    69m   v1.32.9   194.163.147.140 <none>        Ubuntu 24.04.3 LTS   6.8.0-85-generic containerd://1.6.36
```

Creamos un proyecto lo llamamos “ptaller” en nuestro proyecto de Docker directamente



Creamos todo y montamos la imagen y lo subimos al Docker hub

```
✓ jiahaolin69/webtallerclase Built
jiahaolin@vmi2812274:~/jiahaohlc/docker/caronte/proyecto/ptaller/deploy$ docker push jiahaolin69/webtallerclase
Using default tag: latest
The push refers to repository [docker.io/jiahaolin69/webtallerclase]
17ccc3836282: Pushed
5ded322778c3: Mounted from library/httpd
218870188083: Mounted from library/httpd
e323ab599625: Mounted from library/httpd
5f70bf18a086: Mounted from library/httpd
47ecaab1939c: Mounted from library/httpd
e50a58335e13: Mounted from library/httpd
latest: digest: sha256:2a84b07209929cbbf8ec182ea8d246b8edbe72039674ff3995acb080187ebd02 size: 1779
jiahaolin@vmi2812274:~/jiahaohlc/docker/caronte/proyecto/ptaller/deploy$
```

New

Docker Hardened Images are now FREE for every developer. Register for the webinar. →

hub

ExploreMy Hub

Search Docker Hub

CtrlK

J

jiahaolin69

Docker Personal

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Repositories

All repositories within the jiahaolin69 namespace.

Search by repository name

All content

Create a repository

Name	Last Pushed	Contains	Visibility	Scout
jiahaolin69/webtallerclase	1 minute ago	IMAGE	Public	Inactive
jiahaolin69/ubbase	3 months ago	IMAGE	Public	Inactive

1-2 of 2